

Tugas 12: Tugas Praktikum Mandiri 12

Revani – 0110224111 ¹

¹ Teknik Informatika, STT Terpadu Nurul Fikri, Depok

*E-mail: 0110224111@student.nurulfikri.ac.id

Abstract. Pada tugas latihan praktikum 12 ini dilakukan penerapan metode Principal Component Analysis (PCA) pada dataset Breast Cancer. Dataset ini memiliki banyak fitur numerik hasil pemeriksaan medis yang jika langsung digunakan dapat membuat model menjadi kompleks dan berat secara komputasi. Oleh karena itu, PCA digunakan untuk mereduksi jumlah fitur dengan tetap mempertahankan informasi penting di dalam data. Selain itu, pada praktikum ini juga dilakukan perbandingan performa model Support Vector Machine (SVM) tanpa PCA dan dengan PCA. Dari hasil praktikum, dapat diketahui bahwa PCA mampu mengurangi dimensi data secara signifikan dengan penurunan akurasi yang relatif kecil, sehingga PCA dapat menjadi solusi yang baik untuk menangani data berdimensi tinggi.

1. Penjelasan Hasil Latihan Praktikum

Dataset Breast Cancer yang digunakan memiliki lebih dari 30 fitur numerik. Jika semua fitur langsung digunakan, model memang bisa menghasilkan akurasi tinggi, tetapi model menjadi lebih kompleks dan sulit divisualisasikan. Dengan menerapkan PCA, data direduksi menjadi 2 dan 3 komponen utama sehingga pola data dapat divisualisasikan dengan lebih mudah.

Hasil pengujian menunjukkan bahwa model SVM tanpa PCA memiliki akurasi yang sangat tinggi. Namun setelah PCA diterapkan, akurasi hanya sedikit menurun, sementara jumlah fitur berkurang drastis. Hal ini menunjukkan bahwa PCA masih mampu mempertahankan informasi penting dalam data. Visualisasi PCA 2D dan 3D juga menunjukkan bahwa kelas benign dan malignant cukup terpisah, walaupun masih terdapat sedikit overlap.

2. Penjelasan Kode Program

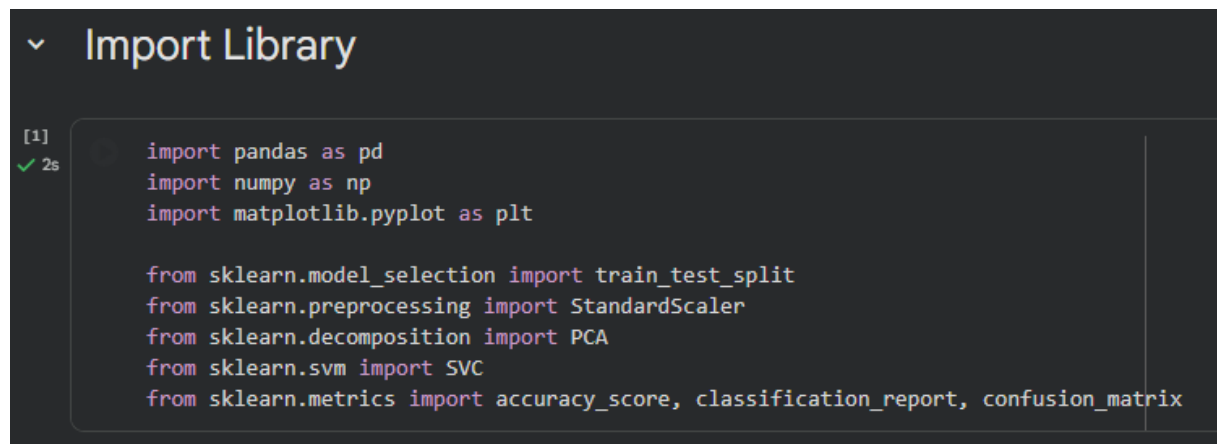
a. Import Library

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
```

```
from sklearn.decomposition import PCA
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score, classification_report,
confusion_matrix
```

Kode ini digunakan untuk memanggil library yang akan digunakan selama praktikum. Pandas dan NumPy digunakan untuk pengolahan data, matplotlib untuk visualisasi, dan scikit-learn untuk preprocessing, PCA, serta pembuatan model SVM seperti yang ditunjukkan pada Gambar 1.



```

[1]
✓ 2s
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix

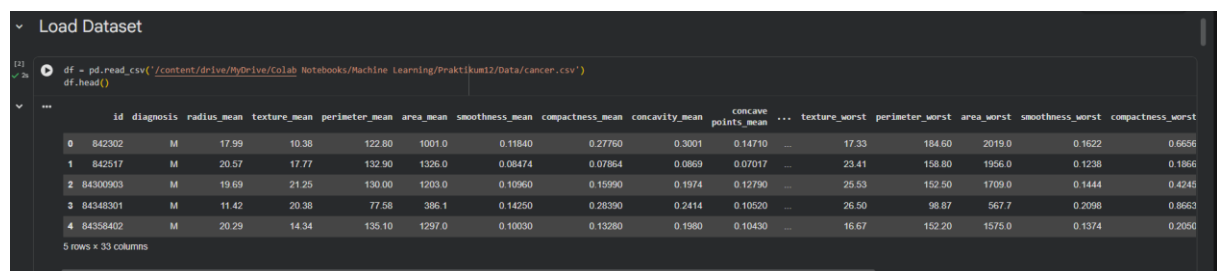
```

Gambar 1. Pada bagian ini ditunjukkan import library.

b. Load Dataset

```
df = pd.read_csv('/content/drive/MyDrive/Colab Notebooks/Machine
Learning/Praktikum12/Data/cancer.csv')
df.head()
```

Kode ini digunakan untuk membaca dataset Breast Cancer dan menampilkan lima baris pertama untuk memastikan data berhasil dimuat. Output yang ditampilkan yaitu beberapa baris data awal yang berisi fitur numerik dan kolom diagnosis seperti yang ditunjukkan pada Gambar 2.



	id	diagnosis	radius_mean	texture_mean	perimeter_mean	area_mean	smoothness_mean	compactness_mean	concavity_mean	concave points_mean	...	texture_worst	perimeter_worst	area_worst	smoothness_worst	compactness_worst
0	842302	M	17.99	10.38	122.80	1001.0	0.11840	0.27760	0.3001	0.14710	...	17.33	184.60	2019.0	0.1622	0.6656
1	842517	M	20.57	17.77	132.90	1326.0	0.08474	0.07864	0.0869	0.07017	...	23.41	158.80	1956.0	0.1238	0.1866
2	84300903	M	19.69	21.25	130.00	1203.0	0.10960	0.15990	0.1974	0.12790	...	25.53	152.50	1709.0	0.1444	0.4245
3	84348301	M	11.42	20.38	77.58	386.1	0.14250	0.28390	0.2414	0.10520	...	26.50	98.87	567.7	0.2098	0.8663
4	84358402	M	20.29	14.34	135.10	1297.0	0.10030	0.13280	0.1980	0.10430	...	16.67	152.20	1575.0	0.1374	0.2050

5 rows x 33 columns

Gambar 2. Pada bagian ini ditunjukkan load dataset.

c. Informasi Dataset

```
df.info()
df.describe()
```

Kode ini digunakan untuk melihat struktur dataset seperti jumlah baris, kolom, tipe data, dan statistik deskriptif. Dari outputnya terlihat bahwa dataset memiliki banyak fitur numerik dan satu kolom target yaitu diagnosis seperti yang ditunjukkan pada Gambar 3 dan Gambar 4.

```
[3]
✓ Os      df.info()

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 569 entries, 0 to 568
Data columns (total 33 columns):
#   Column                                     Non-Null Count  Dtype
---  -
0   id                                         569 non-null    int64
1   diagnosis                                 569 non-null    object
2   radius_mean                             569 non-null    float64
3   texture_mean                             569 non-null    float64
4   perimeter_mean                           569 non-null    float64
5   area_mean                                569 non-null    float64
6   smoothness_mean                          569 non-null    float64
7   compactness_mean                         569 non-null    float64
8   concavity_mean                           569 non-null    float64
9   concave points_mean                      569 non-null    float64
10  symmetry_mean                             569 non-null    float64
11  fractal_dimension_mean                   569 non-null    float64
12  radius_se                                569 non-null    float64
13  texture_se                                569 non-null    float64
14  perimeter_se                              569 non-null    float64
15  area_se                                   569 non-null    float64
16  smoothness_se                             569 non-null    float64
17  compactness_se                             569 non-null    float64
18  concavity_se                              569 non-null    float64
19  concave points_se                         569 non-null    float64
20  symmetry_se                               569 non-null    float64
21  fractal_dimension_se                     569 non-null    float64
22  radius_worst                             569 non-null    float64
23  texture_worst                             569 non-null    float64
24  perimeter_worst                          569 non-null    float64
25  area_worst                                569 non-null    float64
26  smoothness_worst                         569 non-null    float64
27  compactness_worst                        569 non-null    float64
28  concavity_worst                          569 non-null    float64
29  concave points_worst                     569 non-null    float64
30  symmetry_worst                           569 non-null    float64
31  fractal_dimension_worst                  569 non-null    float64
32  Unnamed: 32                              0 non-null      float64
dtypes: float64(31), int64(1), object(1)
memory usage: 146.8+ KB
```

Gambar 3. Pada bagian ini ditunjukkan informasi dataset.

```
(4)
✓ Os      df.describe()
```

	id	radius_mean	texture_mean	perimeter_mean	area_mean	smoothness_mean	compactness_mean	concavity_mean	concave points_mean	symmetry_mean	...	texture_worst	perimeter_worst	area_worst	smoothness_worst	compactness_worst	concavity_worst	concave points_worst	symmetry_worst	fractal_dimension_worst
count	5.690000e+02	569.000000	569.000000	569.000000	569.000000	569.000000	569.000000	569.000000	569.000000	569.000000	...	569.000000	569.000000	569.000000	569.000000	569.000000	569.000000	569.000000	569.000000	569.000000
mean	3.037183e+07	14.127292	19.289649	91.969033	654.889104	0.096360	0.104341	0.080799	0.048919	0.181162	...	25.677223	107.261213	880.583128	0.132369	0.254265	0.132369	0.254265	0.132369	0.254265
std	1.250206e+08	3.524049	4.301036	24.298981	351.914129	0.014064	0.052813	0.079720	0.038803	0.027414	...	6.146258	33.602542	569.356993	0.022832	0.157336	0.022832	0.157336	0.022832	0.157336
min	8.670000e+03	6.981000	9.710000	43.790000	143.500000	0.052630	0.019380	0.000000	0.000000	0.106000	...	12.020000	50.410000	185.200000	0.071170	0.027290	0.071170	0.027290	0.071170	0.027290
25%	8.692100e+05	11.700000	16.170000	75.170000	420.300000	0.086370	0.064920	0.029560	0.020310	0.161900	...	21.000000	84.110000	515.300000	0.116600	0.147200	0.116600	0.147200	0.116600	0.147200
50%	9.060240e+05	13.370000	18.840000	86.240000	551.100000	0.095870	0.092630	0.061540	0.033500	0.179200	...	25.410000	97.660000	686.500000	0.131300	0.211900	0.131300	0.211900	0.131300	0.211900
75%	8.813129e+06	15.780000	21.800000	104.100000	782.700000	0.105300	0.130400	0.130700	0.074000	0.195700	...	29.720000	125.400000	1084.000000	0.146000	0.339100	0.146000	0.339100	0.146000	0.339100
max	9.113205e+08	28.110000	39.280000	188.500000	2501.000000	0.163400	0.345400	0.426800	0.201200	0.304000	...	49.540000	251.200000	4254.000000	0.222500	1.058000	0.222500	1.058000	0.222500	1.058000

8 rows x 32 columns

Gambar 4. Pada bagian ini ditunjukkan statistik deskriptif.

d. Cek Missing Value dan Duplikat

```
df.isnull().sum()  
df.duplicated().sum()
```

Kode ini digunakan untuk mengecek apakah terdapat data kosong dan data yang terduplikasi seperti yang ditunjukkan pada Gambar 5.

The screenshot shows a Jupyter Notebook interface with two code cells. The first cell contains the code `df.isnull().sum()` and its output is a list of 21 features, each with a value of 0, indicating no missing data. The second cell contains the code `df.duplicated().sum()` and its output is `np.int64(0)`, indicating no duplicated rows.

Feature	Count
id	0
diagnosis	0
radius_mean	0
texture_mean	0
perimeter_mean	0
area_mean	0
smoothness_mean	0
compactness_mean	0
concavity_mean	0
concave points_mean	0
symmetry_mean	0
fractal_dimension_mean	0
radius_se	0
texture_se	0
perimeter_se	0
area_se	0
smoothness_se	0
compactness_se	0
concavity_se	0
concave points_se	0
symmetry_se	0
fractal_dimension_se	0
radius_worst	0
texture_worst	0
perimeter_worst	0
area_worst	0
smoothness_worst	0
compactness_worst	0
concavity_worst	0
concave points_worst	0
svmmetr worst	0

Cek Data Duplikat

Code	Output
<code>df.duplicated().sum()</code>	<code>np.int64(0)</code>

Gambar 5. Pada bagian ini ditunjukkan cek missing value dan duplikat.

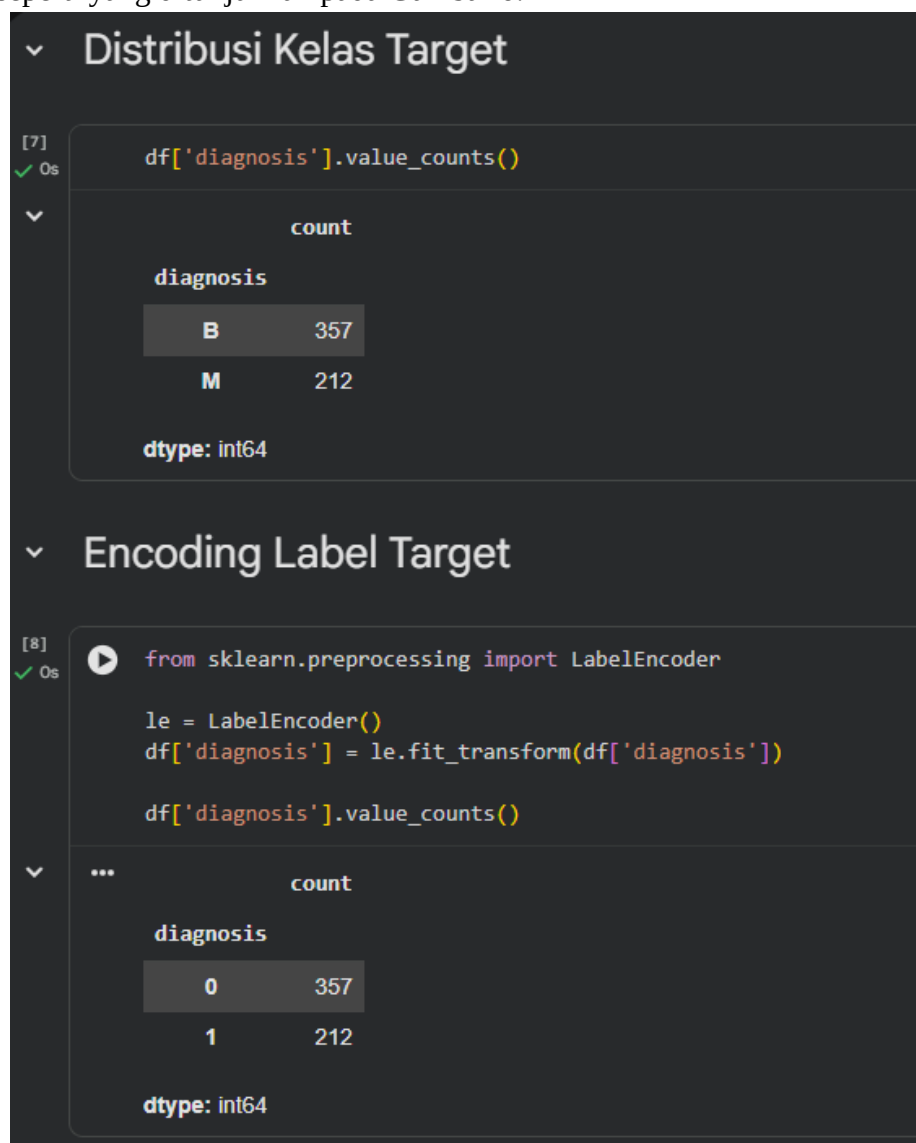
d. Encoding Label Target

```
from sklearn.preprocessing import LabelEncoder

le = LabelEncoder()
df['diagnosis'] = le.fit_transform(df['diagnosis'])

df['diagnosis'].value_counts()
```

Kode ini mengubah label diagnosis dari bentuk huruf menjadi angka agar bisa diproses oleh model machine learning. Outputnya adalah label berubah menjadi 0 dan 1 dengan jumlah data tiap kelas seperti yang ditunjukkan pada Gambar 6.



Gambar 6. Pada bagian ini ditunjukkan encoding label target.

e. Pemisahan Fitur dan Label

```
X = df.drop(['diagnosis', 'Unnamed: 32'], axis=1)
y = df['diagnosis']

print("Shape X:", X.shape)
print("Shape y:", y.shape)
```

Output menunjukkan bahwa X berisi seluruh fitur numerik, sedangkan y hanya berisi satu kolom label seperti yang ditunjukkan pada Gambar 7.



Gambar 7. Pada bagian ini ditunjukkan pemisahan fitur dan label.

f. Train Test Split

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

print("X_train:", X_train.shape)
print("X_test:", X_test.shape)
```

Data dibagi menjadi 80% data latihan dan 20% data uji agar performa model bisa diuji secara objektif. Output yang dihasilkan adalah data latihan dan data uji seperti yang ditunjukkan pada Gambar 8.

```
✓ Train-Test Split (80:20)

[18] ✓ 0s
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

print("X_train:", X_train.shape)
print("X_test:", X_test.shape)

X_train: (455, 31)
X_test: (114, 31)
```

Gambar 8. Pada bagian ini ditunjukkan train test split

g. Standarisasi Data

```
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()

X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

X_train_scaled[:5]
```

Standarisasi dilakukan agar setiap fitur memiliki skala yang sama, karena PCA dan SVM sensitif terhadap perbedaan skala data. Output yang dihasilkan adalah nilai fitur yang sudah terstandarisasi seperti yang ditunjukkan pada Gambar 9.

```
Standardisasi Data

from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()

X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

X_train_scaled[:5]

array([[ -2.43221088e-01,  5.18558727e-01,  8.91825791e-01,
         4.24631702e-01,  3.83925436e-01, -9.74743706e-01,
        -6.89771505e-01, -6.88586446e-01, -3.98175254e-01,
        -1.03915470e+00, -8.25056321e-01, -1.09317755e-01,
        -5.59755400e-02, -2.10096206e-01, -1.59132582e-02,
        -1.00518399e+00, -9.11941990e-01, -6.62815884e-01,
        -6.52561081e-01, -7.01889114e-01, -2.75393571e-01,
         5.79797697e-01,  1.31324246e+00,  4.66908134e-01,
         4.45982711e-01, -5.96154777e-01, -6.34722227e-01,
        -6.10227299e-01, -2.35743918e-01,  5.45663235e-02,
         2.18367276e-02],
       [ 4.08373367e-01, -5.16364088e-01, -1.63971029e+00,
        -5.41348716e-01, -5.42961327e-01,  4.76219058e-01,
        -6.31833818e-01, -6.04281166e-01, -3.03074908e-01,
         5.21543093e-01, -4.54522896e-01, -6.04377961e-01,
        -1.00104604e+00, -5.85429002e-01, -4.93453793e-01,
         4.03212009e-01, -7.68173276e-01, -4.79187222e-01,
         1.14508478e-01, -1.42950761e-01, -5.77397732e-01,
        -5.82458953e-01, -1.69029101e+00, -6.11934288e-01,
        -5.87013537e-01,  2.73581959e-01, -8.14844486e-01,
        -7.12666415e-01, -3.23207881e-01, -1.37576237e-01,
        -9.04401643e-01],
       [-2.42771402e-01, -3.68118388e-01,  4.55514964e-01,
        -3.88249933e-01, -4.02970039e-01, -1.43297929e+00,
        -3.83927370e-01, -3.42175070e-01, -7.65459348e-01,
        -8.50857052e-01, -2.26170901e-01,  3.03979894e-01,
         1.05150064e+00, -1.69545176e-01, -8.09073190e-04,
        -3.10104091e-01,  1.10632991e+00,  6.22584752e-01,
         2.73684564e-01,  7.54482587e-01,  1.50810455e+00,
        -3.98622224e-01,  1.81977388e-01, -4.75431283e-01,
        -4.20778328e-01, -1.62278520e+00, -3.91399175e-01,
        -4.31312898e-01, -8.90825037e-01, -6.75892999e-01,
        -1.44015592e-01],
       [-2.42750834e-01,  2.05284794e-01,  7.26167669e-01,
         4.00330308e-01,  7.06116009e-02,  2.43253484e-01,
         2.20358457e+00,  2.25609405e+00,  1.21323326e+00,
         8.18473995e-01,  8.99791136e-01, -5.45729713e-01,
        -6.21676545e-01,  2.61427407e-01, -3.53584902e-01,
         2.44600975e-02,  2.09072809e+00,  1.49056147e+00,
         1.69512701e+00, -6.54909382e-01,  7.67547750e-01,
        -3.09312591e-04,  2.74191135e-01,  5.13776118e-01,
        -9.94819409e-02,  4.18530802e-01,  2.06596968e+00,
         2.95861933e+00,  1.97706439e+00, -7.56459905e-02,
         1.72884831e+00],
       [-2.43167137e-01,  1.24300470e+00,  1.94195111e-01,
         1.21037678e+00,  1.20665201e+00, -1.11441850e-01,
         5.13480707e-02,  7.32962322e-01,  7.13767250e-01,
        -4.27187350e-01, -8.22183969e-01,  1.52386344e+00,
         1.14394136e+00,  1.28274755e+00,  1.00131268e+00,
         1.35184527e+00,  1.07730938e-01,  5.92926514e-01,
         1.18098825e+00,  3.01549783e-01,  1.71526944e-01,
         1.01283532e+00,  2.23144239e-01,  9.38517229e-01,
         8.80910405e-01,  7.32014364e-02, -2.77005741e-01,
         3.27775115e-01,  5.01858851e-01, -9.09322392e-01,
        -5.46248793e-01]])
```

Gambar 9. Pada bagian ini ditunjukkan standarisasi data.

h. Baseline Model SVM tanpa PCA

```
svm_no_pca = SVC(kernel='rbf', gamma='scale', random_state=42)
svm_no_pca.fit(X_train_scaled, y_train)

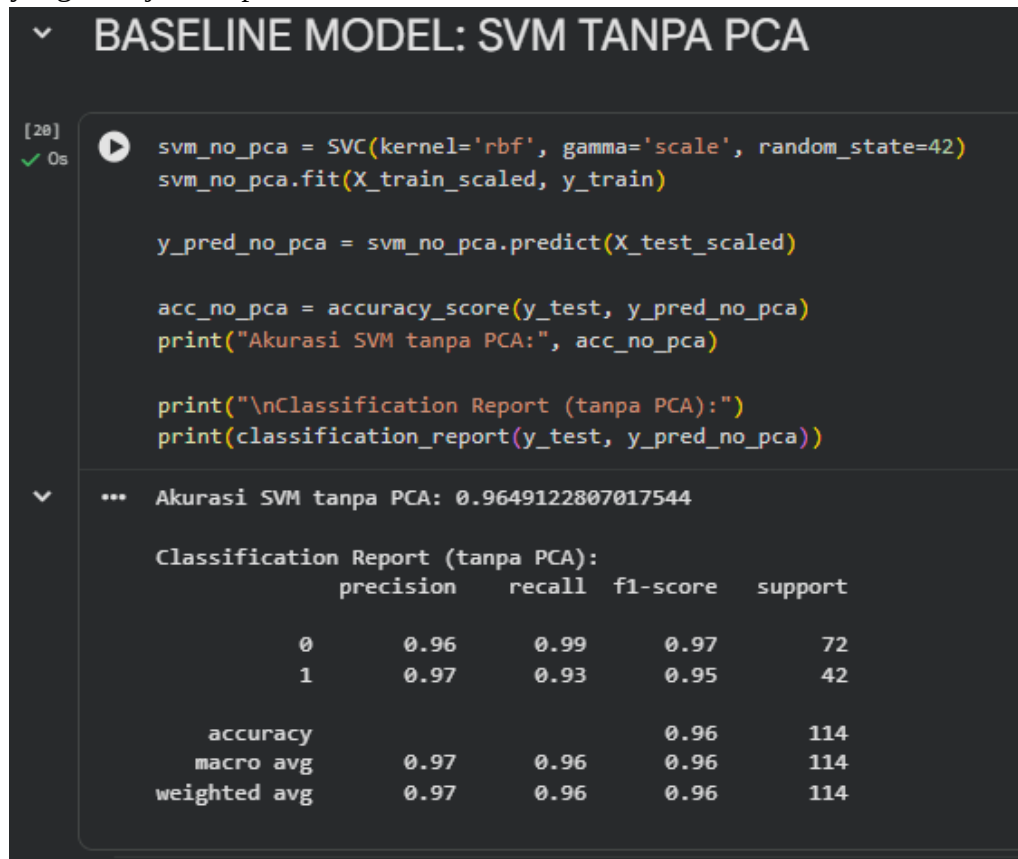
y_pred_no_pca = svm_no_pca.predict(X_test_scaled)

acc_no_pca = accuracy_score(y_test, y_pred_no_pca)
print("Akurasi SVM tanpa PCA:", acc_no_pca)
```



```
print("\nClassification Report (tanpa PCA):")
print(classification_report(y_test, y_pred_no_pca))
```

Kode ini digunakan untuk membuat model SVM tanpa PCA sebagai pembanding awal. Outputnya yaitu akurasi tinggi biasanya di atas 95% dengan nilai precision dan recall yang baik seperti yang ditunjukkan pada Gambar 10.



Gambar 10. Pada bagian ini ditunjukkan SVM tanpa PCA.

i. PCA 2 Dimensi

```
pca = PCA(n_components=2)
X_train_pca = pca.fit_transform(X_train_scaled)
X_test_pca = pca.transform(X_test_scaled)

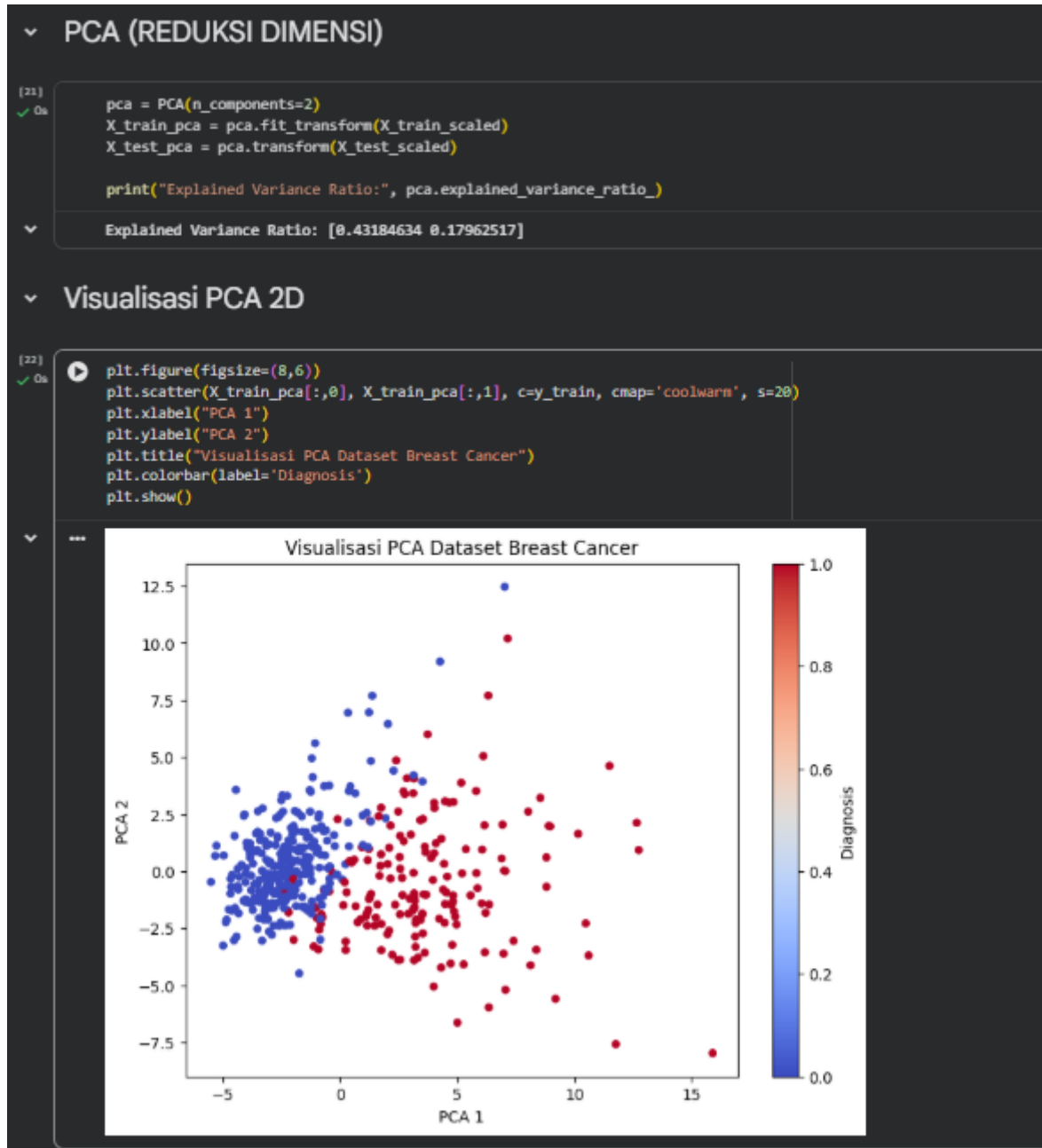
print("Explained Variance Ratio:", pca.explained_variance_ratio_)
```

PCA mereduksi fitur menjadi dua komponen utama. Outputnya nilai explained variance ratio yang menunjukkan proporsi informasi yang dipertahankan.

```
plt.figure(figsize=(8,6))
plt.scatter(X_train_pca[:,0], X_train_pca[:,1], c=y_train,
            cmap='coolwarm', s=20)
plt.xlabel("PCA 1")
plt.ylabel("PCA 2")
```

```
plt.title("Visualisasi PCA Dataset Breast Cancer")
plt.colorbar(label='Diagnosis')
plt.show()
```

Output yang dihasilkan adalah grafik scatter 2D yang menunjukkan pemisahan kelas berdasarkan PCA seperti yang ditunjukkan pada Gambar 11.



Gambar 11. Pada bagian ini ditunjukkan PCA 2D.

j. PCA 3 Dimensi

```
# Apply PCA for 3 components for 3D visualization
pca_3d = PCA(n_components=3)
X_train_pca_3d = pca_3d.fit_transform(X_train_scaled)
X_test_pca_3d = pca_3d.transform(X_test_scaled)

print("Explained Variance Ratio (3 components):")
print(pca_3d.explained_variance_ratio_)
print("Cumulative Explained Variance (3 components):")
print(np.sum(pca_3d.explained_variance_ratio_))
```

PCA mereduksi fitur menjadi tiga komponen utama. Outputnya nilai explained variance ratio dan cumulative explained variance.

```
fig = plt.figure(figsize=(10, 8))
ax = fig.add_subplot(111, projection='3d')

scatter = ax.scatter(
    X_train_pca_3d[:, 0],
    X_train_pca_3d[:, 1],
    X_train_pca_3d[:, 2],
    c=y_train,
    cmap='coolwarm',
    s=40
)

ax.set_xlabel('PCA 1')
ax.set_ylabel('PCA 2')
ax.set_zlabel('PCA 3')
ax.set_title('Visualisasi PCA 3D Dataset Breast Cancer')
plt.colorbar(scatter, label='Diagnosis')
plt.show()
```

Output yang dihasilkan adalah visualisasi PCA 3D yang memperlihatkan distribusi data lebih jelas seperti yang ditunjukkan pada Gambar 12.

Visualisasi PCA 3D

```
[36]: # Apply PCA for 3 components for 3D visualization
      pca_3d = PCA(n_components=3)
      X_train_pca_3d = pca_3d.fit_transform(X_train_scaled)
      X_test_pca_3d = pca_3d.transform(X_test_scaled)

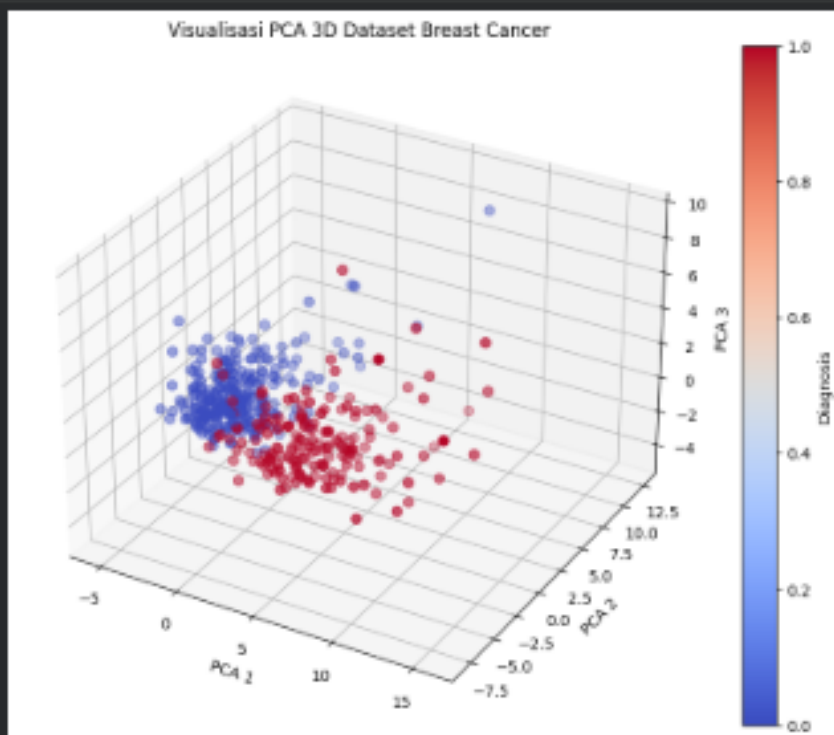
      print("Explained Variance Ratio (3 components):")
      print(pca_3d.explained_variance_ratio_)
      print("Cumulative Explained Variance (3 components):")
      print(np.sum(pca_3d.explained_variance_ratio_))
```

```
Explained Variance Ratio (3 components):
[0.43384634 0.17962517 0.4035514 ]
Cumulative Explained Variance (3 components):
0.705029980608617
```

```
[37]: fig = plt.figure(figsize=(10, 8))
      ax = fig.add_subplot(111, projection='3d')

      scatter = ax.scatter(
          X_train_pca_3d[:, 0],
          X_train_pca_3d[:, 1],
          X_train_pca_3d[:, 2],
          c=y_train,
          cmap='coolwarm',
          s=40
      )

      ax.set_xlabel('PCA 1')
      ax.set_ylabel('PCA 2')
      ax.set_zlabel('PCA 3')
      ax.set_title("Visualisasi PCA 3D Dataset Breast Cancer")
      plt.colorbar(scatter, label='Diagnosis')
      plt.show()
```



Gambar 12. Pada bagian ini ditunjukkan PCA 3D.

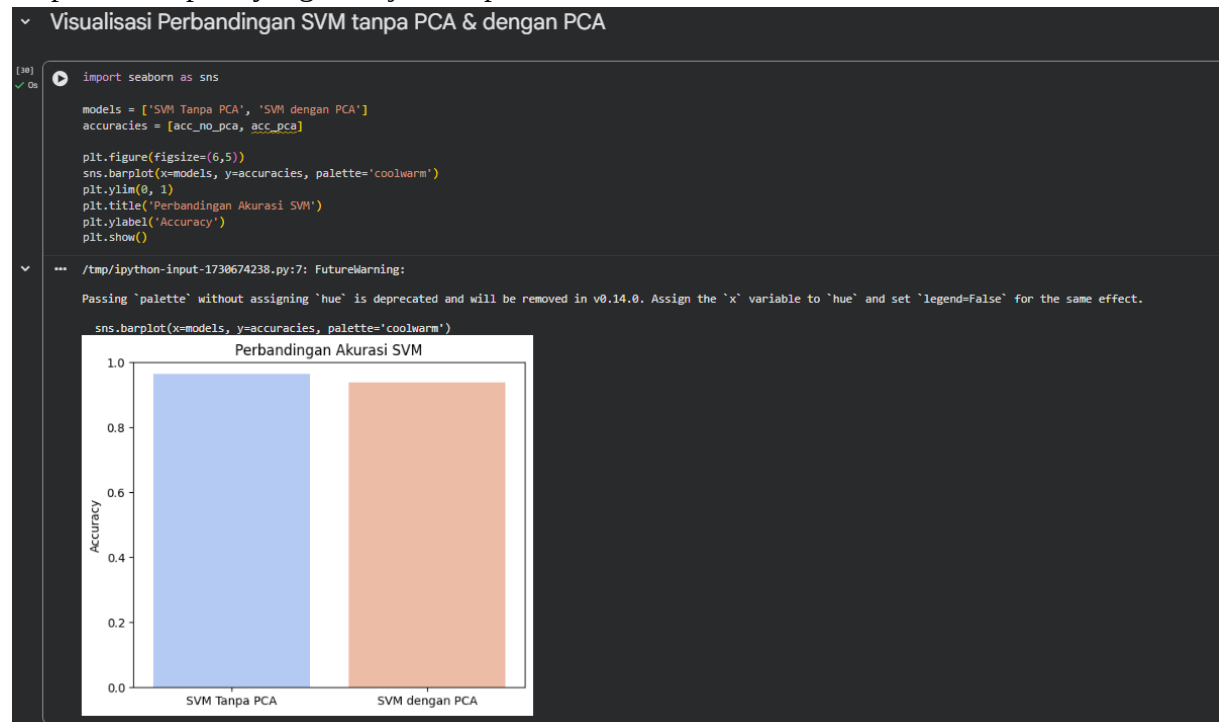
k. Perbandingan Akurasi

```
import seaborn as sns

models = ['SVM Tanpa PCA', 'SVM dengan PCA']
accuracies = [acc_no_pca, acc_pca]

plt.figure(figsize=(6,5))
sns.barplot(x=models, y=accuracies, palette='coolwarm')
plt.ylim(0, 1)
plt.title('Perbandingan Akurasi SVM')
plt.ylabel('Accuracy')
plt.show()
```

Output yang dihasilkan adalah grafik batang perbandingan akurasi antara SVM dengan dan tanpa PCA seperti yang ditunjukkan pada Gambar 13.



Gambar 13. Pada bagian ini ditunjukkan perbandingan akurasi.

3. Kesimpulan dan Hasil Implementasi

Dari seluruh praktikum ini, saya menyimpulkan bahwa PCA dapat diterapkan pada berbagai bidang, terutama pada data medis yang memiliki banyak fitur seperti data hasil pemeriksaan laboratorium atau citra medis. Dengan PCA, data dapat diproses lebih cepat, divisualisasikan dengan lebih mudah, dan tetap mempertahankan informasi penting yang dibutuhkan untuk klasifikasi. PCA sangat berguna untuk mereduksi dimensi data berdimensi tinggi. Walaupun akurasi model sedikit menurun setelah PCA diterapkan, penurunannya tidak signifikan. Dengan jumlah fitur yang jauh lebih sedikit, model menjadi lebih efisien dan lebih mudah dianalisis. Oleh karena itu, PCA merupakan teknik preprocessing yang penting sebelum menerapkan model machine learning pada dataset dengan banyak fitur.

4. Link GitHub (Praktikum dan Latihan Pertemuan 12)

https://github.com/revani18/ML_2025_Revani_3AI01/tree/1160bb42b9f6aaac731e8306a41f7c4482834ec3/Praktikum12

Praktikum:

https://github.com/revani18/ML_2025_Revani_3AI01/blob/1160bb42b9f6aaac731e8306a41f7c4482834ec3/Praktikum12/Notebooks/praktikum12.ipynb

Latihan:

https://github.com/revani18/ML_2025_Revani_3AI01/blob/1160bb42b9f6aaac731e8306a41f7c4482834ec3/Praktikum12/Notebooks/latihan12.ipynb